
Special Topics in AI

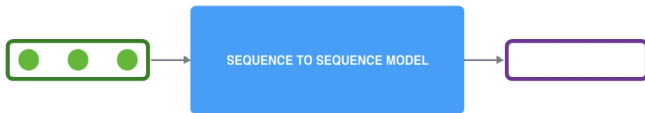
Lecture 2: Attention

University of Mazandaran

Fall 1405

Sequence-to-Sequence (Seq2Seq)

Models that map one sequence of items into another

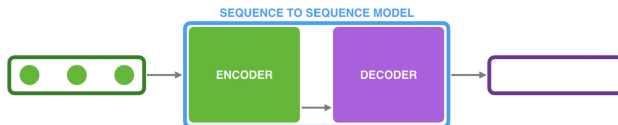


[link to the file](#)

Sequence-to-Sequence (Seq2Seq)

Models that map one sequence of items into another

- ▶ Though, in modern DL literature, "Seq2Seq models" specifically refers to neural encoder-decoder family of models

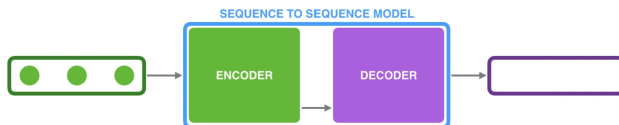


[link to the file](#)

Sequence-to-Sequence (Seq2Seq)

Models that map one sequence of items into another

- ▶ Though, in modern DL literature, "Seq2Seq models" specifically refers to neural encoder-decoder family of models



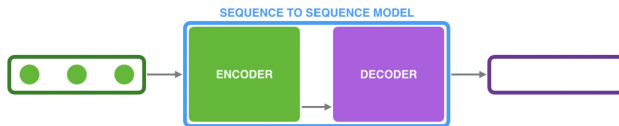
[link to the file](#)

- ▶ Encoder: takes a variable-length sequence of elements as input and transforms it into a context representation with a fixed-size
- ▶ Decoder: maps the context of a fixed-size to a variable-length sequence of elements

Sequence-to-Sequence (Seq2Seq)

Models that map one sequence of items into another

- ▶ Though, in modern DL literature, "Seq2Seq models" specifically refers to neural encoder-decoder family of models



[link to the file](#)

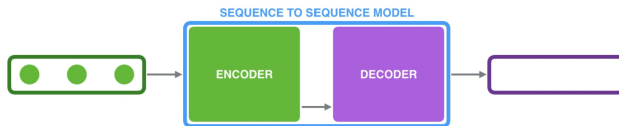
- ▶ Encoder: takes a variable-length sequence of elements as input and transforms it into a context representation with a fixed-size
- ▶ Decoder: maps the context of a fixed-size to a variable-length sequence of elements

Success in machine translation, text summarization and image captioning

Sequence-to-Sequence (Seq2Seq)

Models that map one sequence of items into another

- ▶ Though, in modern DL literature, "Seq2Seq models" specifically refers to neural encoder-decoder family of models



[link to the file](#)

- ▶ Encoder: takes a variable-length sequence of elements as input and transforms it into a context representation with a fixed-size
- ▶ Decoder: maps the context of a fixed-size to a variable-length sequence of elements

Success in machine translation, text summarization and image captioning

Machine translation is the use of computational techniques to translate text or speech from one language to another.

Two important concepts:

1. Alignment: identifying which parts of the input sentence are relevant to each word in the output
2. Translation: using relevant information to select the appropriate output from a set of possible outputs.

Neural Machine Translation With a seq2seq Model

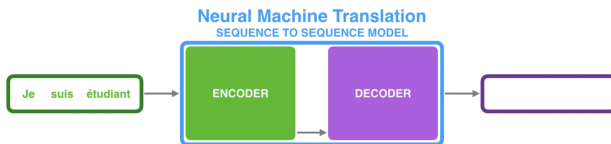
In Neural Machine Translation (NMT), the alignment-translation is done end-to-end using seq2seq model

- ▶ An encoder neural network reads and encodes a source sentence into a fixed-length context vector
- ▶ A decoder then outputs a translation from the encoded context vector

Neural Machine Translation With a seq2seq Model

In Neural Machine Translation (NMT), the alignment-translation is done end-to-end using seq2seq model

- ▶ An encoder neural network reads and encodes a source sentence into a fixed-length context vector
- ▶ A decoder then outputs a translation from the encoded context vector



[link to the file](#)

Defining seq2seq for NMT¹: Encoder

- ▶ An encoder encodes the input sentence, a sequence of vectors $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)$, into a context vector

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

Defining seq2seq for NMT¹: Encoder

- ▶ An encoder encodes the input sentence, a sequence of vectors $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)$, into a context vector
 - ▶ the encoder is typically an RNN/LSTM

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

For a simple RNN $\mathbf{h}_t = \tanh(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + b)$.

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

Defining seq2seq for NMT¹: Encoder

- ▶ An encoder encodes the input sentence, a sequence of vectors $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)$, into a context vector
 - ▶ the encoder is typically an RNN/LSTM

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

For a simple RNN $\mathbf{h}_t = \tanh(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + b)$.

- ▶ Each hidden state summarizes everything seen so far

$$\mathbf{h}_1 = f(\mathbf{h}_0, \mathbf{x}_1) \rightarrow \text{information about } \mathbf{x}_1$$

$$\mathbf{h}_2 = f(\mathbf{h}_1, \mathbf{x}_2) \rightarrow \text{information about } \mathbf{x}_1, \mathbf{x}_2$$

...

$$\mathbf{h}_N = f(\mathbf{h}_{N-1}, \mathbf{x}_N) \rightarrow \text{information about } \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$$

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

Defining seq2seq for NMT¹: Encoder

- ▶ An encoder encodes the input sentence, a sequence of vectors $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)$, into a context vector
 - ▶ the encoder is typically an RNN/LSTM

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

For a simple RNN $\mathbf{h}_t = \tanh(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + b)$.

- ▶ Each hidden state summarizes everything seen so far

$$\mathbf{h}_1 = f(\mathbf{h}_0, \mathbf{x}_1) \rightarrow \text{information about } \mathbf{x}_1$$

$$\mathbf{h}_2 = f(\mathbf{h}_1, \mathbf{x}_2) \rightarrow \text{information about } \mathbf{x}_1, \mathbf{x}_2$$

...

$$\mathbf{h}_N = f(\mathbf{h}_{N-1}, \mathbf{x}_N) \rightarrow \text{information about } \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$$

- ▶ Finally $\mathbf{c} = q(\{\mathbf{h}_1, \dots, \mathbf{h}_N\})$ where
 - ▶ $\mathbf{c}$ is the context vector generated from the hidden states
 - ▶ q is any linear or nonlinear function

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

Defining seq2seq for NMT¹: Decoder

- ▶ The decoder is trained to predict the next word $\mathbf{y}_t$, given the context vector $\mathbf{c}$ and all the previously predicted words $\{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}$

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

Defining seq2seq for NMT¹: Decoder

- ▶ The decoder is trained to predict the next word $\mathbf{y}_t$, given the context vector $\mathbf{c}$ and all the previously predicted words $\{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}$
- ▶ It defines a probability over the translation $\mathbf{y}$ by decomposing the joint probability into the conditionals:

$$p(\mathbf{y}) = \prod_{t=1}^T p(\mathbf{y}_t \mid \{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}, \mathbf{c}), \text{ where } \mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_T)$$

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

Defining seq2seq for NMT¹: Decoder

- ▶ The decoder is trained to predict the next word $\mathbf{y}_t$, given the context vector $\mathbf{c}$ and all the previously predicted words $\{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}$
- ▶ It defines a probability over the translation $\mathbf{y}$ by decomposing the joint probability into the conditionals:

$$p(\mathbf{y}) = \prod_{t=1}^T p(\mathbf{y}_t \mid \{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}, \mathbf{c}), \text{ where } \mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_T)$$

- ▶ With an RNN decoder, each conditional probability is modeled as:

$$p(\mathbf{y}_t \mid \{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}, \mathbf{c}) = g(\mathbf{y}_{t-1}, \mathbf{s}_t, \mathbf{c})$$

- ▶ g is a nonlinear (multi-layered) function
- ▶ $\mathbf{s}_t$ is the hidden state of the RNN

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

The Issue With The seq2seq Models

- ▶ The model needs to compress all necessary information of a source sequence into the fixed-length context vector $\mathbf{c}$
- ▶ It is challenging for the model to compress all information into a fixed-length context, especially when dealing with long sequences.

- ▶ The attention mechanism¹ overcomes the bottleneck issue of encoder-decoder

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

- ▶ The attention mechanism¹ overcomes the bottleneck issue of encoder-decoder
- ▶ Intuitively it allows the model to **focus** on relevant parts of the input, while decoding the output

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

- ▶ The attention mechanism¹ overcomes the bottleneck issue of encoder-decoder
- ▶ Intuitively it allows the model to **focus** on relevant parts of the input, while decoding the output
- ▶ In the context of NMT: Each time the model generates a word, it searches for a set of positions in the source sentence where the most relevant information is concentrated.

¹Neural Machine Translation by Learning to Align and Translate, Bahdanu et al. ICLR (2015)

An intuitive example

In the following sentence, determine the word to which "it" refers.

The dog chased the cat b/c it was scared.

An intuitive example

In the following sentence, determine the word to which "it" refers.

The dog chased the cat b/c it was scared.

Our brain **attends** to different parts of the sentence:

- ▶ we remember "chased" and the roles of the dog and the cat
- ▶ we evaluate which one is more likely to be scared

An intuitive example

In the following sentence, determine the word to which "it" refers.

The dog chased the cat b/c it was scared.

Our brain **attends** to different parts of the sentence:

- ▶ we remember "chased" and the roles of the dog and the cat
- ▶ we evaluate which one is more likely to be scared

This mental weighting of relevant words is what attention does!

- ▶ The encoder-decoder with attention does not need to encode the whole input into a single fixed-length vector

- ▶ The encoder-decoder with attention does not need to encode the whole input into a single fixed-length vector
- ▶ It instead encodes the input into a sequence of vectors

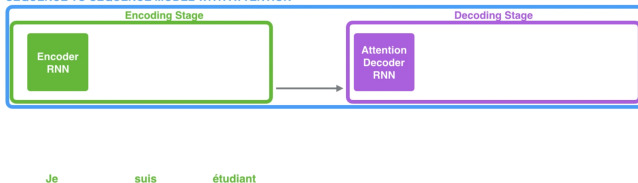
- ▶ The encoder-decoder with attention does not need to encode the whole input into a single fixed-length vector
- ▶ It instead encodes the input into a sequence of vectors
- ▶ Then it choose a subset of these vectors adaptively while decoding the output

Attention

- ▶ The encoder-decoder with attention does not need to encode the whole input into a single fixed-length vector
- ▶ It instead encodes the input into a sequence of vectors
- ▶ Then it choose a subset of these vectors adaptively while decoding the output
- ▶ This frees the model from having to **squash all the information** of the source into a fixed-length vector.

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Step-by-step

1. **Input encoding**: each token (say word!) i from the input sentence $\mathbf{x}_i$ is encoded by encoder to a hidden vector $\mathbf{h}_i$

Step-by-step

1. **Input encoding**: each token (say word!) i from the input sentence $\mathbf{x}_i$ is encoded by encoder to a hidden vector $\mathbf{h}_i$
2. **Alignment scores**: at decoder step t , the model computes an alignment score e_{ti} between the previous decoder state $\mathbf{s}_{t-1}$ and every encoder hidden state $\mathbf{h}_i$
 - ▶ when scoring the first output for the decoder, $\mathbf{s}$ will be a vector of zeros!

Alignment score and alignment function

- ▶ The alignment score e_{ti} indicates how relevant each source position i is when generating the current target token $\mathbf{y}_t$
- ▶ It is computed using a small feed-forward neural network represented by $a(\cdot)$:

$$e_{ti} = a(\mathbf{s}_{t-1}, \mathbf{h}_i) = \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_i)$$

- ▶ The alignment model is trained jointly with all the other components of the model

Step-by-step

1. **Input encoding**: each token (say word!) i from the input sentence $\mathbf{x}_i$ is encoded by encoder to a hidden vector $\mathbf{h}_i$
2. **Alignment scores**: at decoder step t , the model computes an alignment score e_{ti} between the previous decoder state $\mathbf{s}_{t-1}$ and every encoder hidden state $\mathbf{h}_i$
 - ▶ when scoring the first output for the decoder, $\mathbf{s}$ will be a vector of zeros!
3. **Weight calculation**: the weights α_{ti} are computed by applying a softmax operation to the alignment scores

$$\alpha_{ti} = \text{softmax}(e_{ti})$$

Step-by-step

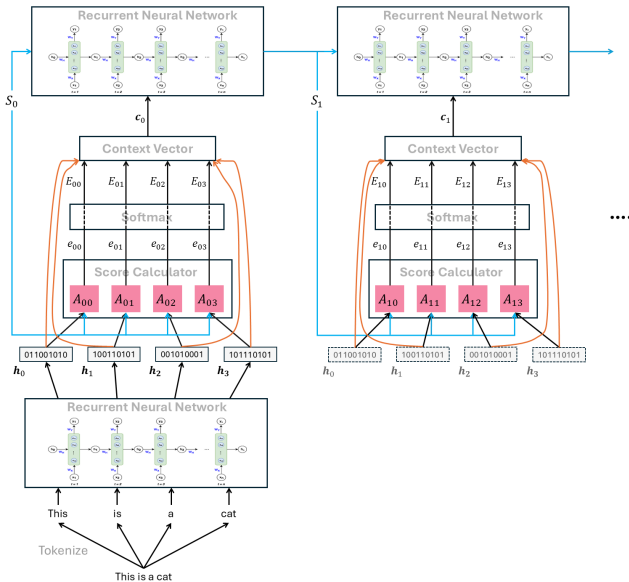
1. **Input encoding**: each token (say word!) i from the input sentence $\mathbf{x}_i$ is encoded by encoder to a hidden vector $\mathbf{h}_i$
2. **Alignment scores**: at decoder step t , the model computes an alignment score e_{ti} between the previous decoder state $\mathbf{s}_{t-1}$ and every encoder hidden state $\mathbf{h}_i$
 - ▶ when scoring the first output for the decoder, $\mathbf{s}$ will be a vector of zeros!
3. **Weight calculation**: the weights α_{ti} are computed by applying a softmax operation to the alignment scores

$$\alpha_{ti} = \text{softmax}(e_{ti})$$

4. **Context vector**: a unique context vector $\mathbf{c}_t$ is generated by a weighted sum of all encoder hidden states weighted by the weights computed in previous step

$$\mathbf{c}_t = \sum_{i=1} \alpha_{ti} \mathbf{h}_i$$

Attention



- ▶ **Issue:** computing the attention score is cumbersome!
 - ▶ say, the encoder sequence is of length m and the decoder sequence is of size n
 - ▶ we require $m \times n$ alignment score calculation (evaluating a small feed-forward network)
 - ▶ the decoder is also an RNN, so to compute s_i , we have to compute s_{i-1} first!
 - ▶ the $m \times n$ alignment score calculation could not be paralleled

Do we really need the recurrent state $\mathbf{s}_i$ at all?

²Attention is All You Need, Vaswani et al., NeurIPS (2017)

Do we really need the recurrent state s_i at all? No!

²Attention is All You Need, Vaswani et al., NeurIPS (2017)

Attention: A More Efficient Version²

1. Pass each input token $\mathbf{x}_i$ through the encoder to get
 $\mathbf{H}^{\text{enc}} = [\mathbf{h}_1^{\text{enc}}, \mathbf{h}_2^{\text{enc}}, \dots, \mathbf{h}_N^{\text{enc}}]$
2. Pass each output token $\mathbf{y}_j$ through the decoder to get
 $\mathbf{H}^{\text{dec}} = [\mathbf{h}_1^{\text{dec}}, \mathbf{h}_2^{\text{dec}}, \dots, \mathbf{h}_T^{\text{dec}}]$
3. Project these representation into three learned spaces: $\mathbf{Q} = \mathbf{H}^{\text{dec}} \mathbf{W}_Q$,
 $\mathbf{K} = \mathbf{H}^{\text{enc}} \mathbf{W}_K$, $\mathbf{V} = \mathbf{H}^{\text{enc}} \mathbf{W}_V$
4. Compute all pairwise score with one matrix multiplication! $\mathbf{E} = \mathbf{QK}^T$
5. Scale the attention scores: $\tilde{\mathbf{E}} = \frac{\mathbf{QK}^T}{\sqrt{d_k}}$
 - ▶ d_k is the dimensionality of the queries and keys
 - ▶ with large d_k , each $e_{ij} = \mathbf{q}_i^T \mathbf{k}_j$ becomes a large value and their softmax become extremely peaked which produces very small gradients
6. Convert scores to attention weights: $\mathbf{A} = \text{softmax}(\tilde{\mathbf{E}})$
7. Generate the output representation: $\mathbf{Z} = \mathbf{AV}$.

²Attention is All You Need, Vaswani et al., NeurIPS (2017)

Attention: A More Efficient Version²

1. Pass each input token $\mathbf{x}_i$ through the encoder to get $\mathbf{H}^{\text{enc}} = [\mathbf{h}_1^{\text{enc}}, \mathbf{h}_2^{\text{enc}}, \dots, \mathbf{h}_N^{\text{enc}}]$
2. Pass each output token $\mathbf{y}_j$ through the decoder to get $\mathbf{H}^{\text{dec}} = [\mathbf{h}_1^{\text{dec}}, \mathbf{h}_2^{\text{dec}}, \dots, \mathbf{h}_T^{\text{dec}}]$
3. Project these representation into three learned spaces: $\mathbf{Q} = \mathbf{H}^{\text{dec}} \mathbf{W}_Q$,
 $\mathbf{K} = \mathbf{H}^{\text{enc}} \mathbf{W}_K$, $\mathbf{V} = \mathbf{H}^{\text{enc}} \mathbf{W}_V$
4. Compute all pairwise score with one matrix multiplication! $\mathbf{E} = \mathbf{Q} \mathbf{K}^T$
5. Scale the attention scores: $\tilde{\mathbf{E}} = \frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{d_k}}$
 - ▶ d_k is the dimensionality of the queries and keys
 - ▶ with large d_k , each $e_{ij} = \mathbf{q}_i^T \mathbf{k}_j$ becomes a large value and their softmax become extremely peaked which produces very small gradients
6. Convert scores to attention weights: $\mathbf{A} = \text{softmax}(\tilde{\mathbf{E}})$ ← $\mathbf{A}$ is called the Attention Matrix
7. Generate the output representation: $\mathbf{Z} = \mathbf{A} \mathbf{V}$.

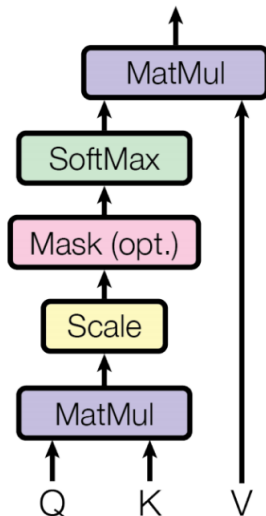
²Attention is All You Need, Vaswani et al., NeurIPS (2017)

The General Attention Mechanism

Makes use of 3 main components: **Queries**, **Keys**, and **Values**.
This is similar to information retrieval systems:

- ▶ Search for a video on Youtube!
- ▶ **Query**: text in the search bar
- ▶ **Keys**: video titles, descriptions, etc., associated with candidate videos
- ▶ **Values**: best matched videos

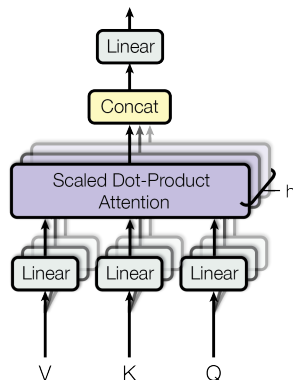
The General Attention Mechanism



Multi-Head Attention

What if we want to perform multiple attention operations in parallel?

- ▶ For each attention head h :
 $\mathbf{Q}_h = \mathbf{H}^{\text{dec}} \mathbf{W}_h^Q, \mathbf{K}_h = \mathbf{H}^{\text{enc}} \mathbf{W}_h^K, \mathbf{V}_h = \mathbf{H}^{\text{enc}} \mathbf{W}_h^V$
- ▶ Each head performs scaled dot-product attention: $\text{head}_h = \text{softmax} \left(\frac{\mathbf{Q}_h \mathbf{K}_h^T}{\sqrt{d_k}} \right) \mathbf{V}_h$
- ▶ Outputs of all heads are concatenated
 $\text{Concat}(\text{head}_1, \dots, \text{head}_H)$
- ▶ Concatenated heads are passed through another learned linear projection
 $\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}^O$



Intuition

Consider a football team consist of 5 players:

- ▶ 2 strikers
- ▶ 1 midfielder
- ▶ 1 defender
- ▶ 1 goalkeeper.

Players have to understand their role in their context (team).

Intuition

Consider a football team consist of 5 players:

- ▶ 2 strikers
- ▶ 1 midfielder
- ▶ 1 defender
- ▶ 1 goalkeeper.

Players have to understand their role in their context (team).

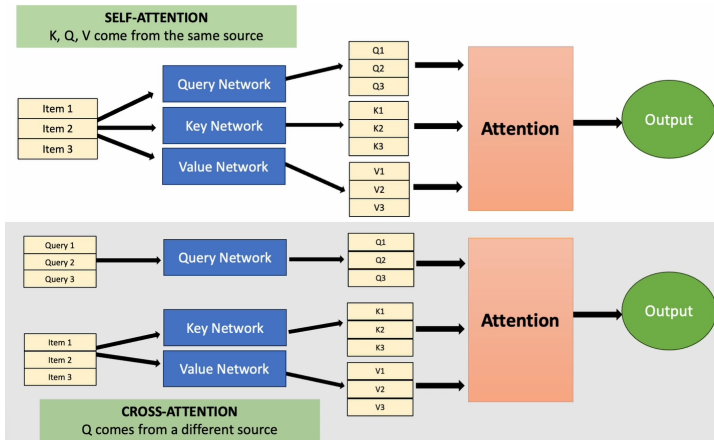
- ▶ Self-attention tells striker A to focus (put the attention) more on striker B and then the midfielder
- ▶ Self-attention tells goalkeeper to focus (put the attention) more on defenders than the striker
- ▶ And so on!

Self-Attention (a.k.a. **Intra-Attention**) tells the players how much attention they should put on their team-mates,

Attention (a.k.a. **Cross-Attention**) tells the players how much attention they should put on rival players

Self-Attention

All the queries, keys, and values come from the same latent place, e.g., the latent representation of the input tokens to the encoder.



Self-Attention

